Digital Banking Platform – Microservices Roadmap

By: Backend roadmap for a 2.9 YOE engineer building a production-grade portfolio project

1. Goal and Positioning

Build a **realistic digital banking backend** that proves you can handle what product and fintech companies actually hire for: money-safe transactions, high concurrency, event-driven design, security, and observability. The React frontend and the RAG/AI layer come after the backend is solid.

What this project should let you say in an interview:

- I designed a double-entry ledger that never loses or duplicates money under concurrent load.
- I used the Outbox pattern + Kafka for reliable event delivery, and idempotent consumers for exactly-once effects.
- I implemented a Saga for distributed transfers with compensation.
- I secured everything with Keycloak (OAuth2/OIDC, RBAC) behind an API Gateway.
- I load-tested it (k6/Gatling) and tuned it with real numbers.

Rule of thumb: depth beats breadth. A smaller number of services done properly (tests, docs, load tests, diagrams) beats 15 half-built ones.

2. Tech Stack

Area	Choice
Language / Framework	Java 21, Spring Boot 3.x, Spring Cloud
Auth	Keycloak (OAuth2 / OIDC, JWT, RBAC)
Gateway	Spring Cloud Gateway (rate limiting, JWT validation, routing)
Service discovery / config	Kubernetes-native (or Eureka + Spring Cloud Config for local)

Area	Choice
Messaging	Apache Kafka (+ Schema Registry, Avro or JSON Schema)
Databases	PostgreSQL (one DB per service), Redis (cache, locks, rate limit), Elasticsearch/OpenSearch (search, optional)
Migrations	Flyway
Resilience	Resilience4j (circuit breaker, retry, bulkhead, rate limiter)
Sync communication	REST (OpenFeign / RestClient), optional gRPC for ledger
Observability	Micrometer, Prometheus, Grafana, Loki or ELK, OpenTelemetry + Jaeger/Tempo
Testing	JUnit 5, Mockito, Testcontainers, WireMock, Pact (contract), k6/Gatling (load)
Build / Deploy	Maven or Gradle multi-module, Docker, Docker Compose, then Kubernetes (kind/minikube), GitHub Actions CI
API docs	springdoc-openapi (Swagger) per service, aggregated at gateway
Frontend (later)	React + TypeScript, Vite, TanStack Query, Keycloak JS / OIDC client
AI (later)	Spring AI, pgvector, LLM provider, RAG pipeline

3. High-Level Architecture

Request flow: React → API Gateway → Microservices → PostgreSQL / Redis / Kafka.

Auth flow: Client logs in at Keycloak (Authorization Code + PKCE) → receives JWT → Gateway validates JWT → forwards to services → each service re-validates as a resource server and applies role/ownership checks.

Event flow: Service writes business data + outbox row in the same DB transaction → outbox relay publishes to Kafka → consumers process idempotently → results published as new events.

Services

#	Service	Responsibility	DB
1	api-gateway	Routing, JWT check, rate limiting, CORS, request logging, correlation ID	Redis
2	auth / identity integration (Keycloak)	Realm, clients, roles, user federation	Keycloak DB
3	user-service	Customer profile, KYC status, addresses, preferences	Postgres
4	kyc-service	Document upload, verification workflow, status events	Postgres + object storage (MinIO)
5	account-service	Account creation, types, status, limits, balance read model	Postgres
6	ledger-service	**Core**: double-entry ledger, immutable journal, balances	Postgres
7	transaction-service	Transfer orchestration (Saga), idempotency, status tracking	Postgres
8	payment-service	Bill pay, UPI/IMPS-style simulation, beneficiary, external rails mock	Postgres
9	card-service	Virtual cards, block/unblock, limits, card transactions	Postgres
10	loan-service	Loan products, EMI calculation, applications, repayment schedule	Postgres
11	notification-service	Email/SMS/push via Kafka consumers, templates, retries	Postgres + Redis

#	Service	Responsibility	DB
12	fraud-service	Rule engine on transaction events, risk score, flag/block	Postgres + Redis
13	audit-service	Immutable audit trail of every sensitive action	Postgres / Elasticsearch
14	statement-service	Monthly statements (PDF/CSV), async generation	Postgres + MinIO
15	reporting / analytics-service	Read models via CQRS, dashboards	Postgres / Elasticsearch
16	scheduler-service (or inside services)	Standing instructions, recurring payments, EMI auto-debit, interest accrual	Postgres (ShedLock)

Build order recommendation: start with 1, 3, 5, 6, 7, 11, 13. Add the rest incrementally. Do not start all 16 at once.

4. Feature List per Service (What You Must Build)

4.1 API Gateway

- Route configuration per service, path versioning (/api/v1/...)
- JWT validation against Keycloak JWKS
- Rate limiting per user/IP using Redis (token bucket)
- Correlation ID / trace ID injection on every request
- Global CORS config, security headers
- Circuit breaker + fallback responses
- Request/response logging with PII masking
- Aggregated Swagger UI

4.2 Keycloak (Identity)

- Realm: bank-realm; clients: web-app (public, PKCE), gateway/backend (confidential), service accounts for service-to-service calls
- Roles: CUSTOMER, TELLER, RELATIONSHIP_MANAGER, COMPLIANCE_OFFICER, ADMIN

- Realm export/import as JSON so the setup is reproducible
- MFA / OTP (TOTP) enabled for login and for high-value transactions (step-up auth)
- Token lifetimes, refresh token rotation, session limits
- Custom claims (customerId) via protocol mapper
- Method-level security with @PreAuthorize and ownership checks (a user can only see their own accounts)
- Brute-force protection and account lockout

4.3 User Service

- Register (via Keycloak admin API) and sync profile
- Profile CRUD, address, nominee, contact verification (email/phone OTP)
- KYC status tracking (NOT_STARTED, PENDING, VERIFIED, REJECTED)
- Events: UserRegistered, KycStatusChanged, ProfileUpdated
- Soft delete, GDPR-style data export

4.4 KYC Service

- Document upload (stored in MinIO/S3, metadata in DB)
- Verification workflow with maker-checker (teller submits, compliance approves)
- State machine for status changes
- Emits KycApproved so account-service can activate accounts

4.5 Account Service

- Open savings/current/fixed-deposit accounts (only after KYC verified)
- Unique account number generation (Luhn-like check digit, not sequential-guessable)
- Account states: PENDING, ACTIVE, FROZEN, DORMANT, CLOSED with a state machine
- Daily/monthly limits, minimum balance rules
- Balance read model (updated from ledger events)
- Joint accounts, nominee
- Beneficiary management (add/verify with cooling period)

4.6 Ledger Service (the heart)

- **Double-entry bookkeeping:** every movement is a journal entry with at least two lines (debit and credit) that sum to zero
- Immutable, append-only ledger tables (no UPDATE/DELETE on entries; corrections via reversal entries)
- Chart of accounts: customer accounts, bank settlement account, fee income, suspense/clearing
- Balance computation: running balance column + periodic reconciliation against the sum of entries
- Pending/holds (authorization hold, then capture or release)
- Idempotent posting API keyed by transaction reference
- Multi-currency support (optional, with FX rates table)
- Daily reconciliation job: sum(debits) == sum(credits), ledger vs account-service balance
- Interest accrual and fee posting

4.7 Transaction Service

- Fund transfer types: internal (same bank), interbank (mocked NEFT/IMPS/UPI), scheduled, recurring
- **Idempotency-Key header** support: same key returns the same result, never double-processes
- Transfer Saga (orchestrated): validate → fraud check → debit hold → credit → confirm; compensate on failure
- Transaction state machine: INITIATED → VALIDATED → PENDING → COMPLETED / FAILED / REVERSED
- Limits and velocity checks (daily limit, per-transaction limit)
- Transaction history with cursor pagination, filters, search
- Dispute / reversal flow
- Timeouts and retry with backoff, dead-letter handling for stuck transactions

4.8 Payment Service

- Bill payments, mobile recharge, merchant payments (simulated biller APIs via WireMock)
- Payee/biller directory

- Webhook receiver for external rail callbacks (signature verification, replay protection)
- Payment retry and reconciliation with external provider

4.9 Card Service

- Virtual debit card issuance (masked PAN, tokenized storage; never store raw CVV)
- Block/unblock, set limits, enable/disable online/international usage
- Card authorization endpoint (used by simulated merchant) → ledger hold
- Card statements

4.10 Loan Service

- Loan products (personal, home, auto), eligibility check, EMI calculator
- Application workflow with approval states and maker-checker
- Repayment schedule generation (amortization table)
- Auto-debit EMI via scheduler, overdue detection, penalty posting
- Prepayment and foreclosure

4.11 Notification Service

- Kafka consumers for TransactionCompleted, LowBalance, LoginAlert, CardBlocked, etc.
- Channels: email (SMTP/MailHog), SMS (mock), push (mock), in-app
- Template engine, user preferences, rate limiting and de-duplication
- Retry topics + DLQ

4.12 Fraud Service

- Consumes TransactionInitiated events, returns decision (ALLOW / REVIEW / BLOCK)
- Rules: velocity (N txns in X minutes), unusual amount, new beneficiary + high amount, geo/device mismatch, blacklisted accounts
- Redis sliding-window counters for fast checks
- Case management for REVIEW decisions (analyst UI later)
- Later: ML anomaly scoring (see AI section)

4.13 Audit Service

- Consumes all domain events + security events

- Append-only store, tamper-evidence via hash chaining
- Search by user, entity, date, action
- Retention policy

4.14 Statement Service

- Async generation request → Kafka → worker generates PDF/CSV → stored in MinIO → notification with signed URL
- Monthly auto-statements via scheduler

4.15 Reporting / Analytics

- CQRS read models: spending by category, monthly summary, top merchants
- Admin dashboards: volume, failure rate, fraud rate, p95 latency
- Materialized views or Elasticsearch

5. Core Engineering Topics (The Part That Impresses)

5.1 Transactions and Data Consistency

- ACID transactions inside a service (@Transactional with correct propagation/isolation; understand READ COMMITTED vs REPEATABLE READ vs SERIALIZABLE)
- **No distributed transactions (no 2PC).** Use Saga + Outbox instead
- **Transactional Outbox:** business row + outbox row in one DB transaction; relay via polling publisher or Debezium CDC
- **Idempotent consumers:** processed_events table with unique (consumer, event_id)
- **Saga (orchestration):** transaction-service as orchestrator, with compensating actions (reverse hold, reverse entry)
- Reconciliation jobs that detect and repair drift between services
- Eventual consistency explained in the API (status endpoints, polling or WebSocket/SSE updates)

5.2 Concurrency and Race Conditions

Build explicit tests and demos for each:

Problem	Technique
Two debits at the same time overdraw an account	Pessimistic lock (SELECT … FOR UPDATE) on the account row, or optimistic locking with @Version and retry
Same request sent twice (retry, double click)	Idempotency key + unique constraint
Deadlocks in A→B and B→A simultaneous transfers	Always lock accounts in a consistent order (lowest ID first)
Hot account (merchant/settlement account) contention	Sharded sub-accounts, batched posting, or append-only entries with periodic balance rollup
Duplicate scheduled jobs across instances	ShedLock or Quartz clustered mode
Distributed critical sections	Redis distributed lock (Redisson) with TTL and fencing, used sparingly
Lost updates in read-modify-write	Atomic SQL updates (UPDATE … SET balance = balance - ? WHERE balance >= ?)
Kafka duplicate/reordered messages	Partition key = accountId for ordering, idempotent handlers

Also cover: thread pools, virtual threads (Java 21), @Async with bounded executors, connection pool tuning (HikariCP), backpressure.

5.3 Kafka Design

- Topics: transaction.initiated, transaction.completed, transaction.failed, ledger.entry.posted, account.balance.updated, kyc.status.changed, notification.requested, audit.event, plus retry and DLQ topics
- Partition key strategy (accountId) for per-account ordering
- Producer: acks=all, enable.idempotence=true, retries with backoff
- Consumer: manual commit after successful processing, concurrency tuning, error handler with retry topics and DLT
- Schema Registry with versioning and backward-compatible evolution
- Event envelope standard: eventId, eventType, version, occurredAt, correlationId, payload

- Consumer lag monitoring and alerts
- Optional: Kafka Streams for fraud velocity or real-time aggregates

5.4 Resilience

- Timeouts on every outbound call, retries with jitter (only on idempotent operations), circuit breakers, bulkheads, fallbacks
- Graceful shutdown, health/readiness/liveness probes
- Chaos tests: kill a service mid-transfer and prove no money is lost or duplicated

5.5 Security (Bank-Grade Mindset)

- OAuth2/OIDC with Keycloak, least-privilege roles, ownership checks on every resource (prevent IDOR)
- Input validation, parameterized queries, output encoding
- Secrets in Vault or Kubernetes secrets, never in Git
- Encryption at rest for sensitive columns (AES-GCM, key rotation), TLS everywhere (mTLS between services is a stretch goal)
- PII masking in logs, audit trail for every sensitive read/write
- Rate limiting, brute-force protection, replay protection on webhooks
- Dependency scanning (OWASP Dependency-Check, Trivy), SAST (SonarQube), OWASP Top 10 checklist
- Maker-checker for high-risk operations

5.6 Observability

- Structured JSON logs with traceId/correlationId
- Metrics: request rate, error rate, latency percentiles, Kafka lag, DB pool usage, business metrics (transfers/sec, failure %)
- Distributed tracing across gateway, services, and Kafka (OpenTelemetry)
- Grafana dashboards + alert rules (Prometheus Alertmanager)

5.7 Testing Strategy

- Unit tests for domain logic (target 80%+ on ledger and transaction)
- Integration tests with Testcontainers (Postgres, Kafka, Redis, Keycloak)
- Contract tests (Pact or Spring Cloud Contract) between services

- Concurrency tests (ExecutorService + CountDownLatch firing 100 parallel debits; assert final balance exactly right)
- Property-based tests for ledger invariants (jqwik): total debits == total credits
- Load tests (k6/Gatling): e.g. 500 TPS transfers, measure p95/p99, error rate
- Chaos / failure injection tests

5.8 DevOps

- Dockerfile per service (multi-stage, non-root), docker-compose for the whole stack
- GitHub Actions: build, test, scan, build image, push
- Kubernetes manifests or Helm charts, HPA, resource limits, ConfigMaps/Secrets
- Environment profiles (dev/test/prod), 12-factor config
- Database migrations with Flyway, backward-compatible changes

6. Key Data Model Notes

- **accounts:** id, account_number, customer_id, type, currency, status, version, created_at
- **ledger_accounts:** id, owner_ref, type (CUSTOMER / SETTLEMENT / FEE / SUSPENSE), currency, balance, version
- **journal_entries:** id, transaction_ref (unique), type, status, created_at
- **ledger_lines:** id, journal_entry_id, ledger_account_id, direction (DEBIT/CREDIT), amount (NUMERIC(19,4)), balance_after, created_at
- **transactions:** id, idempotency_key (unique), from_account, to_account, amount, status, failure_reason, created_at
- **outbox_events:** id, aggregate_type, aggregate_id, event_type, payload, status, created_at
- **processed_events:** consumer_name, event_id (composite unique)
- Money rule: **BigDecimal or integer minor units only, never double/float.**

7. Phased Roadmap

Assume roughly 8 to 10 hours per week. Adjust durations to your pace. Each phase ends with a demo-able, tested, documented increment.

Phase 0 – Foundation and Planning (Week 1)

- Write a short PRD: scope, personas, non-goals
- Draw architecture diagrams (C4 context + container, sequence diagram for transfer)
- Define API contracts (OpenAPI) and event contracts for core flows
- Set up monorepo (or multi-repo) with parent POM, shared libs (common-events, common-security, common-observability)
- Docker Compose with Postgres, Kafka, Zookeeper/KRaft, Redis, Keycloak, Prometheus, Grafana, MailHog
- Coding standards, branching strategy, CI skeleton
- **Deliverable:** repo skeleton, docs/ folder, docker-compose up works

Phase 1 – Identity, Gateway, User Service (Weeks 2–3)

- Keycloak realm, clients, roles, realm export
- API Gateway with JWT validation, routing, rate limiting
- User-service with profile CRUD and Keycloak integration
- Resource server security in services, @PreAuthorize, ownership checks
- Global exception handling, validation, standard error format (RFC 7807 Problem Details)
- Flyway, Testcontainers integration tests
- **Deliverable:** log in via Keycloak, call secured APIs through the gateway

Phase 2 – Accounts and KYC (Weeks 4–5)

- Account-service: open account, state machine, limits, beneficiaries
- KYC-service: document upload, maker-checker approval
- First Kafka events (KycApproved → account activation) with the Outbox pattern
- Idempotent consumer framework as a reusable library
- **Deliverable:** end-to-end onboarding: register → KYC → account active

Phase 3 – Ledger Service (Weeks 6–8) ★ most important

- Double-entry model, journal posting API, holds, reversals
- Balance integrity via locking strategy; document why you chose it
- Idempotent posting
- Concurrency test: 200 parallel debits and credits, balance is exactly correct

- Property-based test of ledger invariants
- Reconciliation job
- **Deliverable:** ledger that is provably correct under load

Phase 4 – Transaction Service and Saga (Weeks 9–11)

- Internal transfer with Idempotency-Key
- Orchestrated Saga with compensation
- State machine, timeouts, retries, DLQ
- Limits and velocity checks
- Status API and transaction history with cursor pagination
- Failure-injection tests (kill ledger mid-saga, duplicate requests, out-of-order events)
- **Deliverable:** money transfers that survive failures

Phase 5 – Notifications, Audit, Fraud (Weeks 12–14)

- Notification-service with retry topics and DLQ
- Audit-service with hash-chained append-only log
- Fraud-service with rule engine and Redis counters, integrated synchronously (with fallback) or asynchronously in the saga
- **Deliverable:** every transfer triggers a notification, an audit trail, and a fraud decision

Phase 6 – Payments, Cards, Scheduler (Weeks 15–17)

- Payment-service with mocked billers, webhooks, reconciliation
- Card-service: virtual cards, limits, authorization → ledger hold → capture
- Scheduler: recurring transfers, standing instructions, ShedLock
- **Deliverable:** richer banking products on top of the same ledger

Phase 7 – Loans, Statements, Reporting (Weeks 18–20)

- Loan-service: EMI, amortization, auto-debit, overdue/penalty
- Statement-service: async PDF/CSV generation
- Reporting: CQRS read models and admin metrics
- **Deliverable:** full product surface

Phase 8 – Hardening and Production Readiness (Weeks 21–23)

- Resilience4j everywhere, bulkheads, timeouts
- Full observability: tracing, dashboards, alerts
- Load tests with documented results (TPS, p95, bottlenecks found and fixed)
- Security review against OWASP checklist, dependency scanning
- Kubernetes deployment with HPA, Helm chart
- CI/CD pipeline, SonarQube quality gate
- **Deliverable:** a benchmark report and an architecture decision record (ADR) set

Phase 9 – React Frontend (Weeks 24–27)

- React + TypeScript, Keycloak OIDC login (PKCE), token refresh
- Customer app: dashboard, accounts, transfer flow with step-up OTP, beneficiaries, transaction history with filters, cards, loans, statements, notifications
- Admin / back-office app: KYC review queue, fraud case review, audit search, system metrics
- Real-time updates through SSE/WebSocket for transaction status
- Error handling, loading states, idempotency-key generation on the client, accessibility, responsive design
- **Deliverable:** usable end-to-end demo and a recorded walkthrough video

Phase 10 – AI and RAG Features (Weeks 28+)

See section 9.

8. Suggested Repository Structure

```
digital-bank/
├── docs/
|   ├── architecture/ (C4 diagrams, sequence diagrams)
|   ├── adr/ (architecture decision records)
|   ├── api/ (OpenAPI specs)
|   ├── events/ (event catalog, schemas)
|   └── runbooks/ (incident, DLQ replay, reconciliation)
├── libs/
|   ├── common-events/
|   ├── common-security/
```

```
|   ├── common-outbox/
|   ├── common-idempotency/
|   └── common-observability/
├── services/
|   ├── api-gateway/
|   ├── user-service/
|   ├── kyc-service/
|   ├── account-service/
|   ├── ledger-service/
|   ├── transaction-service/
|   ├── payment-service/
|   ├── card-service/
|   ├── loan-service/
|   ├── notification-service/
|   ├── fraud-service/
|   ├── audit-service/
|   ├── statement-service/
|   └── reporting-service/
├── infra/
|   ├── docker-compose.yml
|   ├── keycloak/ (realm-export.json)
|   ├── k8s/ or helm/
|   └── monitoring/ (Prometheus, Grafana dashboards)
├── load-tests/ (k6 / Gatling scripts)
├── frontend/ (React apps, later)
└── README.md
```

Per-service layout: a clean/hexagonal structure (api, application, domain, infrastructure) so the domain logic is testable without Spring.

9. Later: AI and RAG Features

Add these only after the core platform is stable. They should be a separate **ai-service** (or a few) that consume events and expose APIs, so the banking core is never blocked by AI latency.

9.1 RAG: Banking Assistant / Support Copilot

- Knowledge base: product FAQs, fee schedules, loan policies, RBI/regulatory guidelines (public docs), internal SOPs

- Pipeline: ingestion → chunking → embeddings → vector store (pgvector or Qdrant) → retrieval (hybrid: BM25 + vector) → re-ranking → LLM answer with citations
- Spring AI for orchestration; metadata filters (product type, effective date)
- Guardrails: answer only from retrieved context, refuse out-of-scope questions, no PII leakage, prompt-injection defenses
- Evaluation: a golden question set, groundedness and answer relevance metrics (RAGAS-style), regression tests on prompt/model changes

9.2 Personal Finance Assistant (Customer-Facing)

- Natural language queries: "How much did I spend on food last month?" → LLM converts intent to a safe, parameterized, read-only query via tool/function calling (never raw SQL from the model against production tables)
- Auto-categorization of transactions (merchant text → category) with a classifier or LLM + cache
- Spending insights, budgets, savings suggestions, bill reminders
- Strict per-user data scoping enforced by the backend, not by the prompt

9.3 Fraud and Anomaly Detection (ML)

- Features from transaction history (amount z-score, velocity, new-payee flags, time-of-day)
- Isolation Forest or gradient boosting model trained offline (Python), served via a small model service or ONNX in Java
- Combine with rule engine: rules for hard blocks, model score for REVIEW
- Feedback loop from analyst decisions; drift monitoring

9.4 Agentic Back-Office Helpers

- Compliance assistant: summarizes KYC documents, flags inconsistencies (human approves)
- Fraud analyst copilot: explains why a transaction was flagged using event history
- Support agent assist: drafts replies with retrieved policy citations
- Dispute summarization and routing
- Tool-calling with strict allow-lists, human-in-the-loop for any state-changing action, full audit of every AI action

9.5 AI Platform Concerns

- AI gateway layer: model routing, timeouts, fallbacks, caching, cost and token metrics
- Redaction of PII before sending to external LLMs (or use a self-hosted model)
- Observability for prompts, latencies, tokens, and quality scores
- Rate limits per user, abuse protection

10. Non-Negotiable Checklist (Definition of Done)

- [ ] No floating-point money anywhere
- [ ] Every write endpoint that moves money is idempotent
- [ ] Ledger is append-only and balanced (debits == credits) with a test proving it
- [ ] Concurrency tests pass: 100+ parallel operations produce the exact expected balance
- [ ] Outbox + idempotent consumers on all Kafka flows; DLQ and replay runbook exist
- [ ] Saga compensation tested with injected failures
- [ ] All endpoints secured; ownership checks tested (IDOR tests)
- [ ] Correlation ID visible in logs across all services and in traces
- [ ] Load test results documented with graphs and findings
- [ ] Testcontainers-based integration tests run in CI
- [ ] README with architecture diagram, setup in one command, demo script
- [ ] ADRs for major decisions (why outbox, why this locking strategy, why orchestration over choreography)

11. Common Mistakes to Avoid

- Starting 15 services before one flow works end to end. Build a **thin vertical slice** (register → open account → transfer → notification) first, then widen.
- Sharing a database between services.
- Using Kafka for everything. Use synchronous calls where the user needs an immediate answer, events where decoupling matters.
- Skipping tests for the ledger and transfer logic.
- Over-engineering with Kubernetes and service mesh before the domain logic is correct.
- Not writing documentation. Diagrams and ADRs are what make this project look senior.

12. How to Present It (Resume and Interview)

Resume bullets (examples to earn, not to copy blindly):

- Designed a double-entry ledger microservice in Spring Boot/PostgreSQL handling X TPS with zero balance discrepancies under concurrent load.
- Implemented Saga + Transactional Outbox over Kafka for exactly-once transfer effects, with idempotent consumers and DLQ replay.
- Secured services with Keycloak (OIDC, RBAC, step-up MFA) behind Spring Cloud Gateway with Redis rate limiting.
- Reduced p95 transfer latency from A ms to B ms through connection pool tuning, indexing, and batching (load tested with k6).

Interview topics this project prepares you for: ACID vs eventual consistency, isolation levels, optimistic vs pessimistic locking, idempotency, outbox/CDC, saga vs 2PC, Kafka partitioning/ordering/delivery semantics, circuit breakers, OAuth2/OIDC flows, API security, observability, load testing, system design of a payment system.

13. Suggested Next Step

Start Phase 0 this week: create the repo, the docker-compose stack, and the architecture diagrams. Then build the thin vertical slice through Keycloak, gateway, user-service, account-service, ledger-service, and transaction-service before adding anything else.